

LLM Architectures & Training

Shreya Havaladar

Announcements

- HW 1 is on the website and it is due in 2 weeks!
- Late days are rounded up
 - *1 hr late or 23 hrs late = 1 late day*

Auditing

- This class will likely be offered again!
- You can come to class + participate in discussions and activities
- You can attend OH with questions
- If you do the HWs/project, they will not be graded

Today's Lecture

1. **Neural Networks**
2. Training
3. The Transformer

Neural networks are high-dimensional functions

- A neural network is a high-dimensional function $f(\Theta, x) \rightarrow y$
 - Goal: Map inputs to outputs.
 - LLM-specific output: A probability distribution over a vocabulary.
- Consist of:
 - Parameters (learned weights + biases)
 - Input, hidden, output layers
 - Activation functions

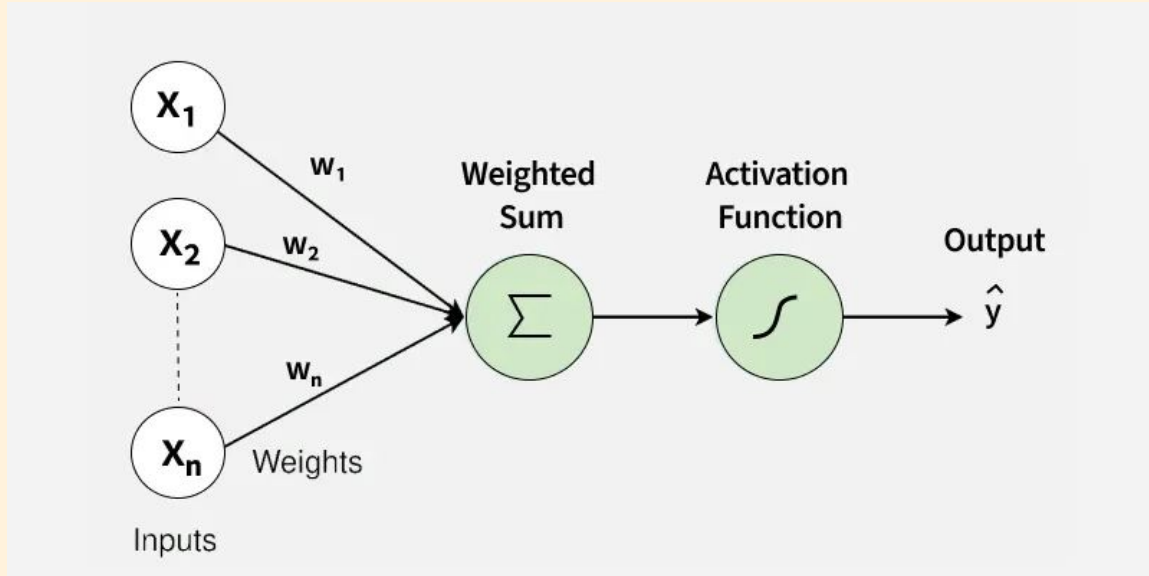
Parameters

Parameters store the knowledge acquired during training and do not change while the model is being used.

Weights: Numerical values representing the strength of connections between neurons, adjusted during learning

Biases: Additional parameters that help shift the activation function, giving the network more flexibility.

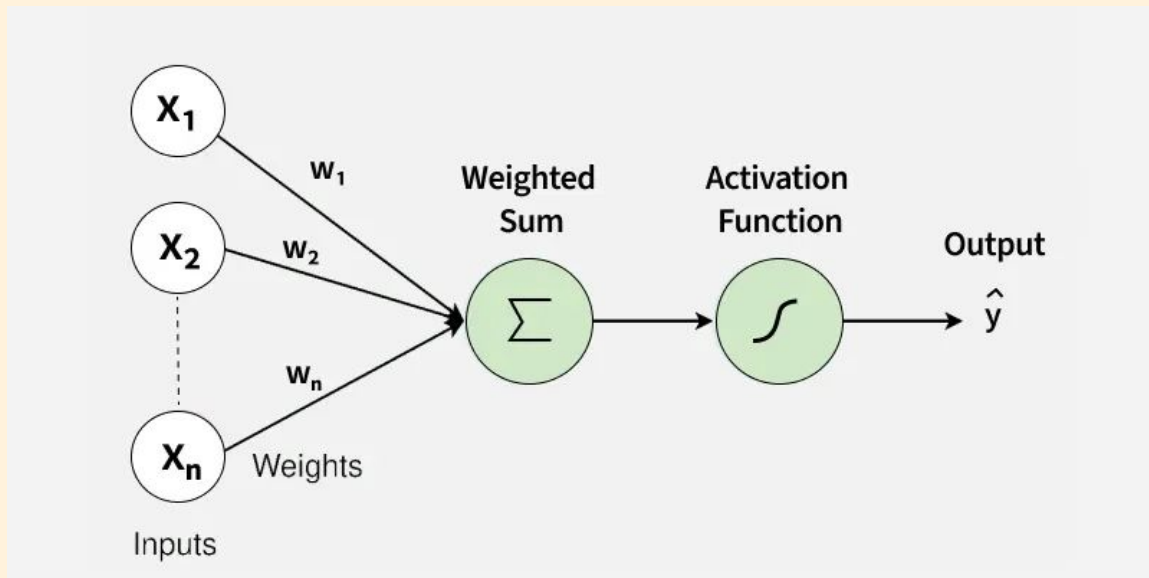
Example: Perceptron



Input: $\langle 3, 5, 4 \rangle$

Weights: $\langle 2, 1, 2 \rangle$

Example: Perceptron



Input: $\langle 3, 5, 4 \rangle$

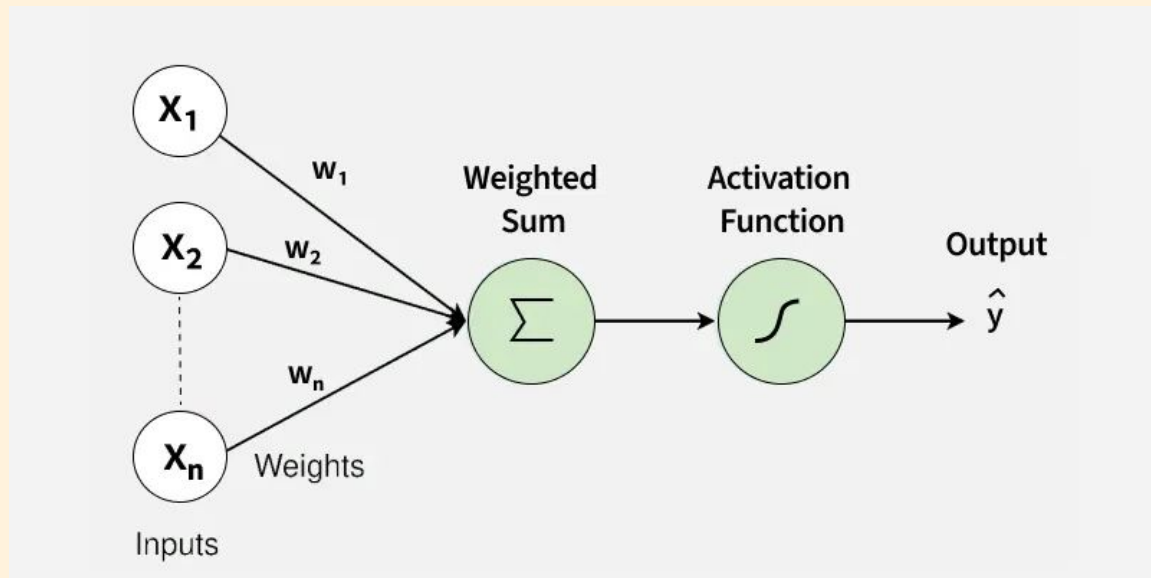
Weights: $\langle 2, 1, 2 \rangle$

Weighted sum:

$$3*2 + 5*1 + 4*2$$

$$= \mathbf{19}$$

Example: Perceptron

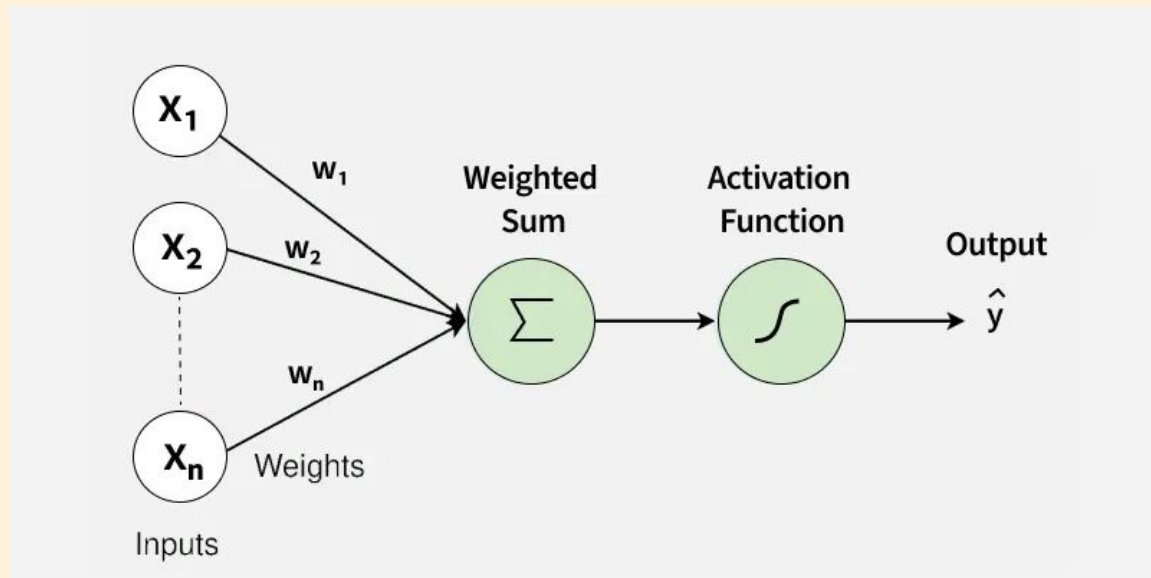


Input: $\langle 3, 5, 4 \rangle$

Weights: $\langle 2, 1, 2 \rangle$

Bias: 1.5

Example: Perceptron



Input: $\langle 3, 5, 4 \rangle$

Weights: $\langle 2, 1, 2 \rangle$

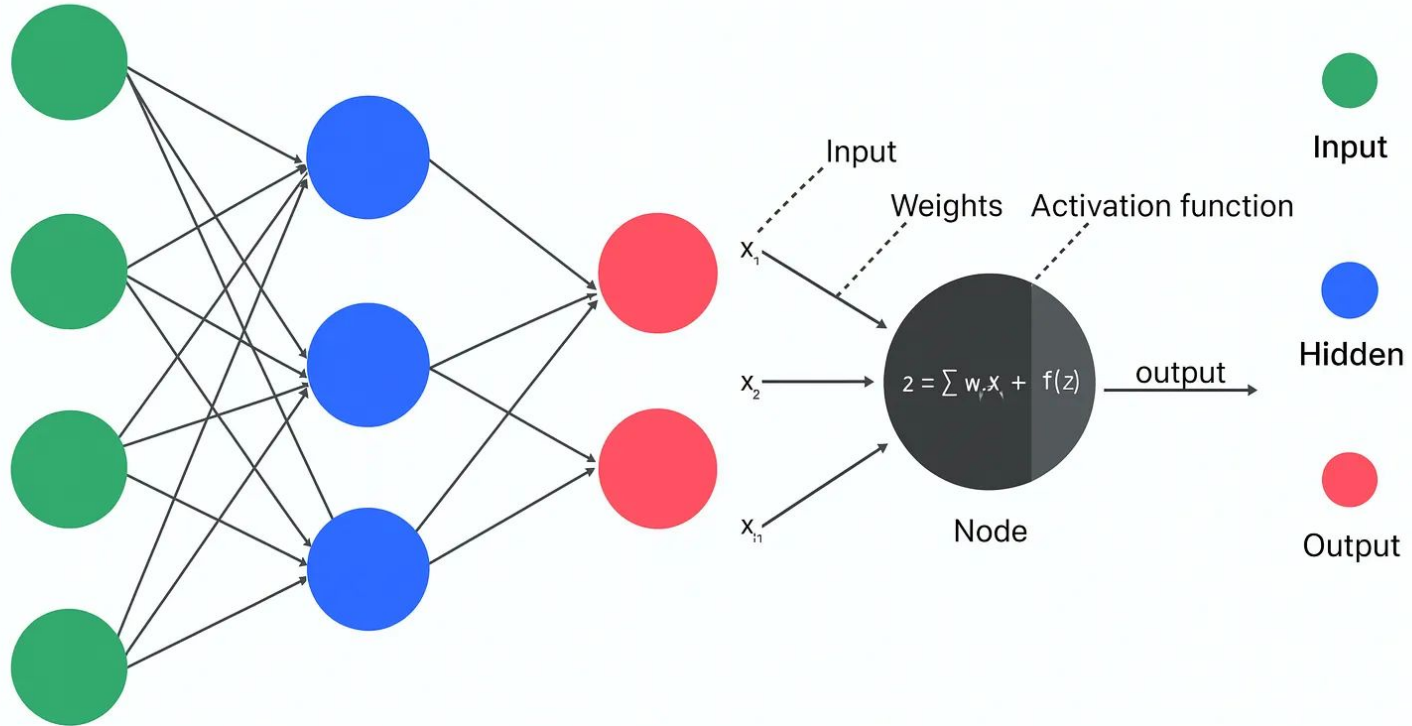
Bias: 1.5

Weighted sum:

$$3 * 2 + 5 * 1 + 4 * 2 + 1.5$$

$$= 20.5$$

Neural Network Architecture



Discuss how many input and output nodes each example network would have

Example 1:

Predict if a student will pass or fail an exam given hours studied, attendance %, and GPA

Example 2:

Given an 28x28 image of a handwritten digit, classify what number it is

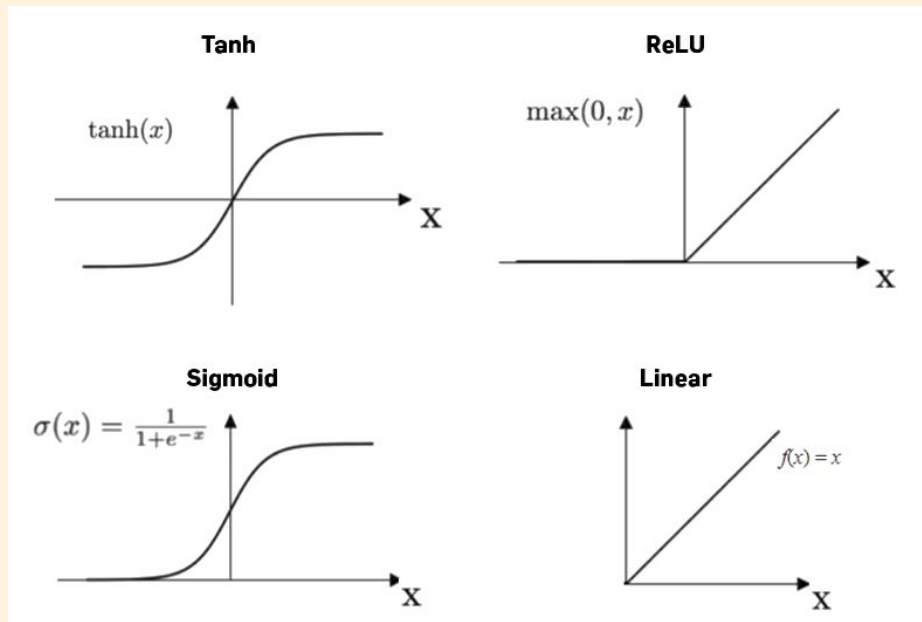
Example 3:

Predicting the next word in a sentence

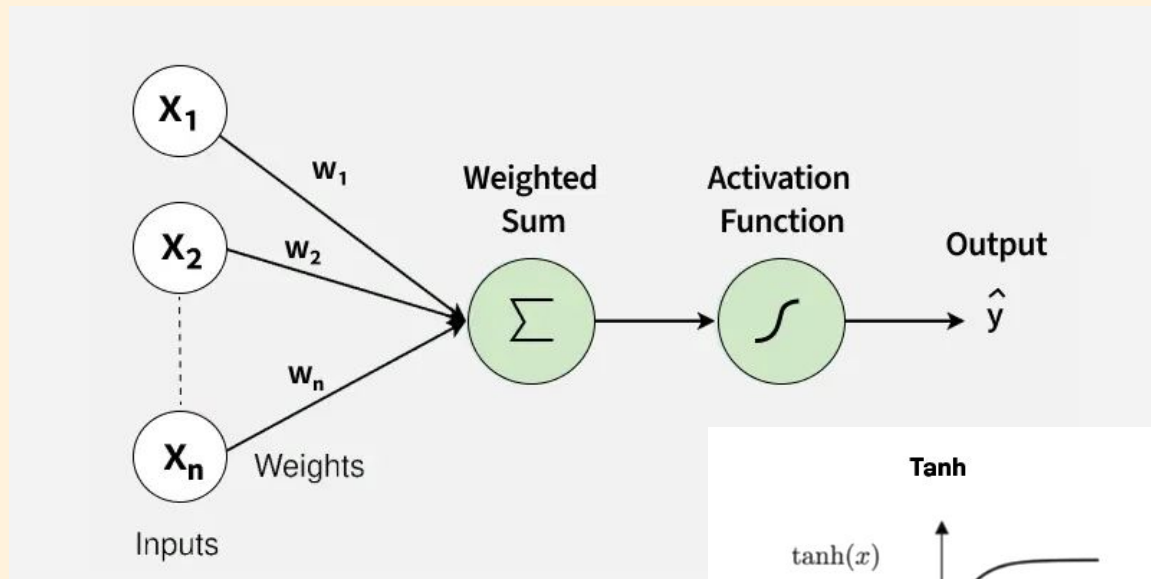
Activation Functions

A mathematical function that decides whether a neuron should "fire" (activate) and pass information to the next layer

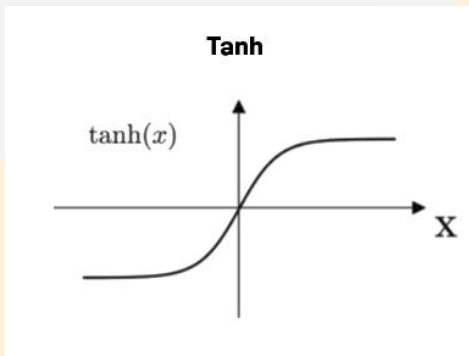
- Introduces **non-linearity**
- Changes with every different input



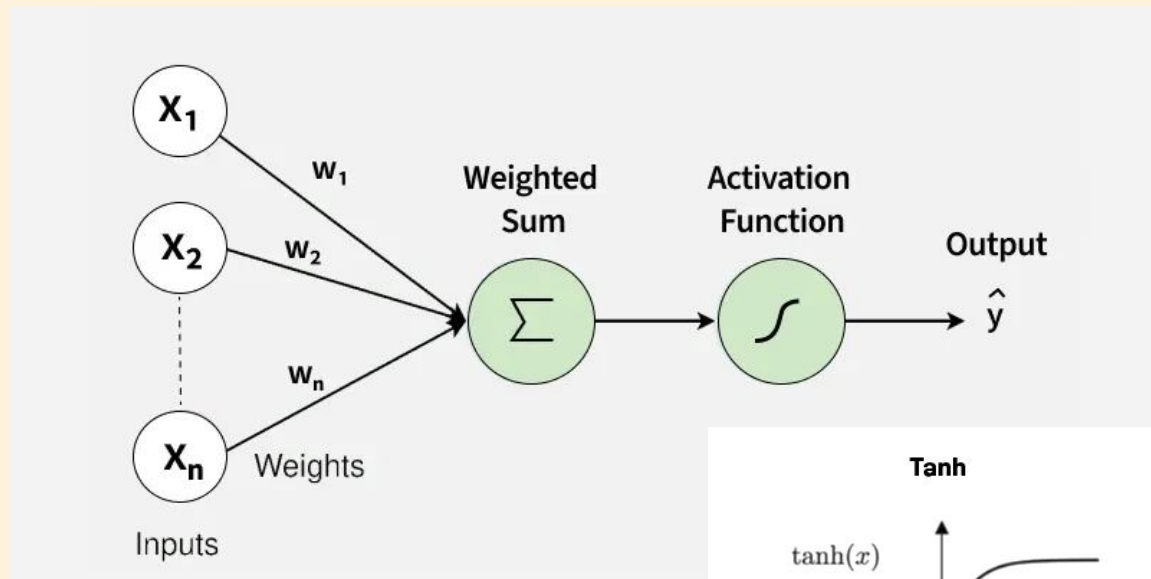
Example: Perceptron



Input: $\langle 3, 5, 4 \rangle$
Weights: $\langle 2, 1, 2 \rangle$
Bias: -22
Activation: Tanh

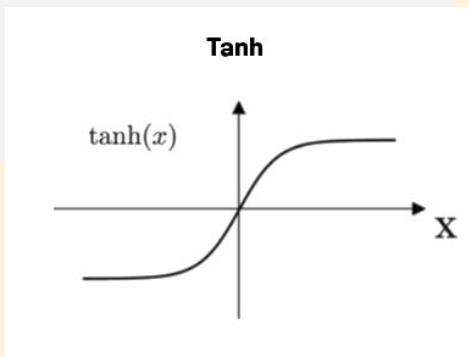


Example: Perceptron

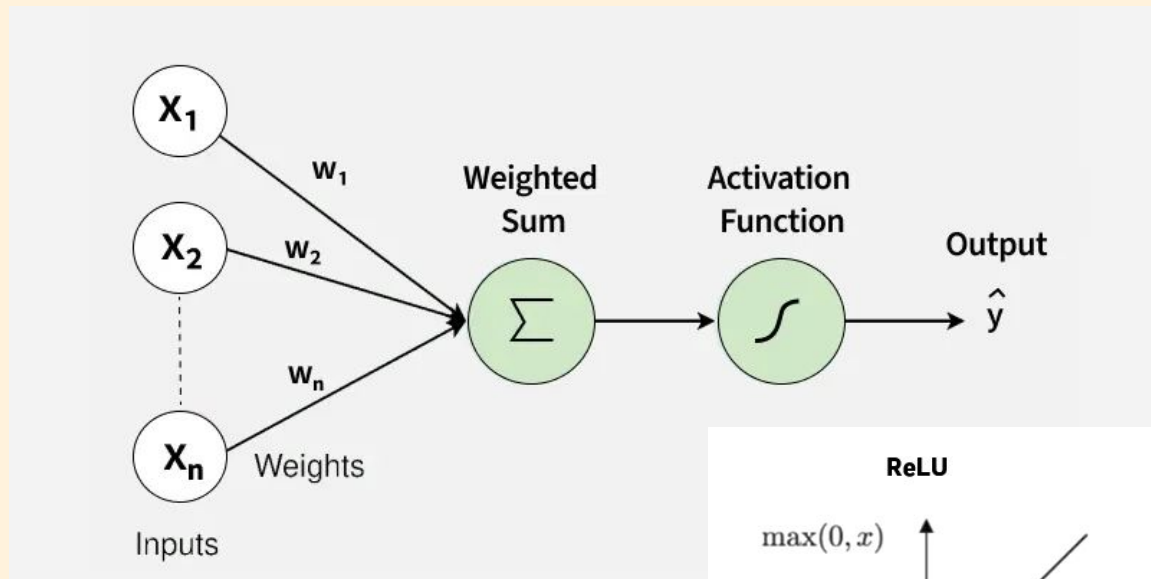


Input: $\langle 3, 5, 4 \rangle$
Weights: $\langle 2, 1, 2 \rangle$
Bias: -22
Activation: Tanh

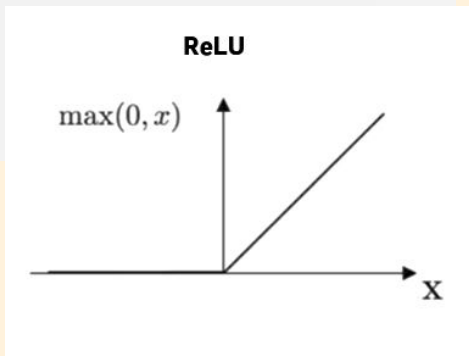
Weighted sum: -3
 $\tanh(-3) = -0.995$



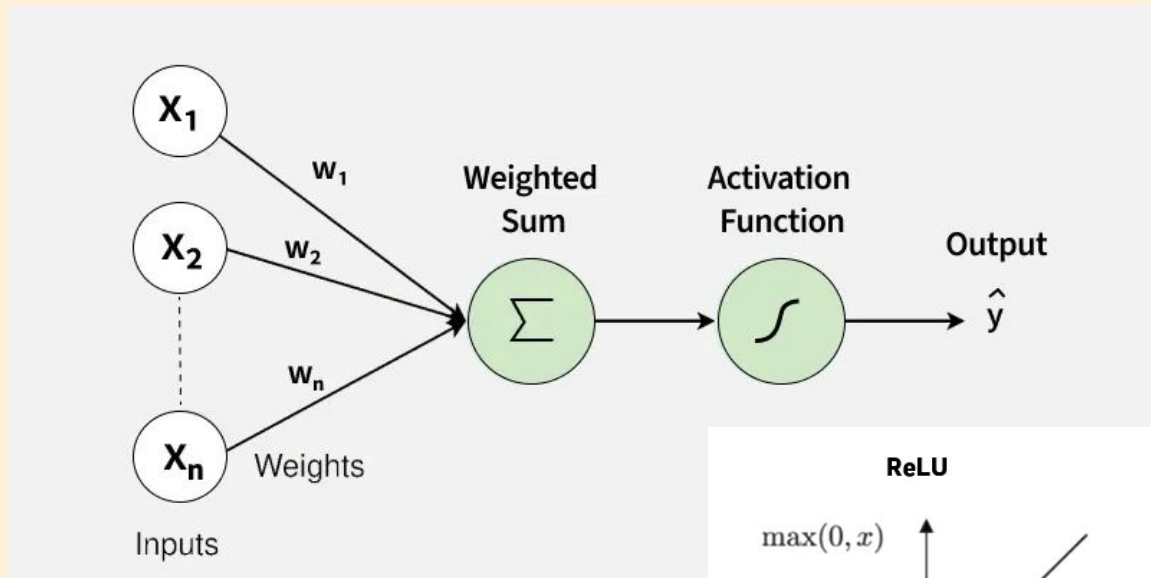
Example: Perceptron



Input: $\langle 3, 5, 4 \rangle$
Weights: $\langle 2, 1, 2 \rangle$
Bias: -22
Activation: ReLU

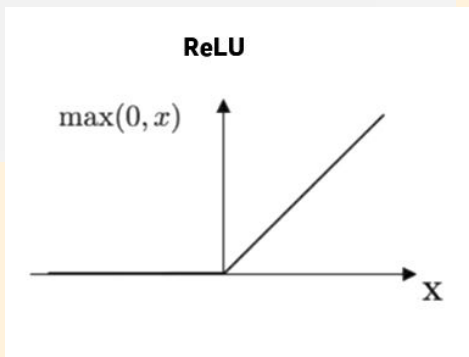


Example: Perceptron

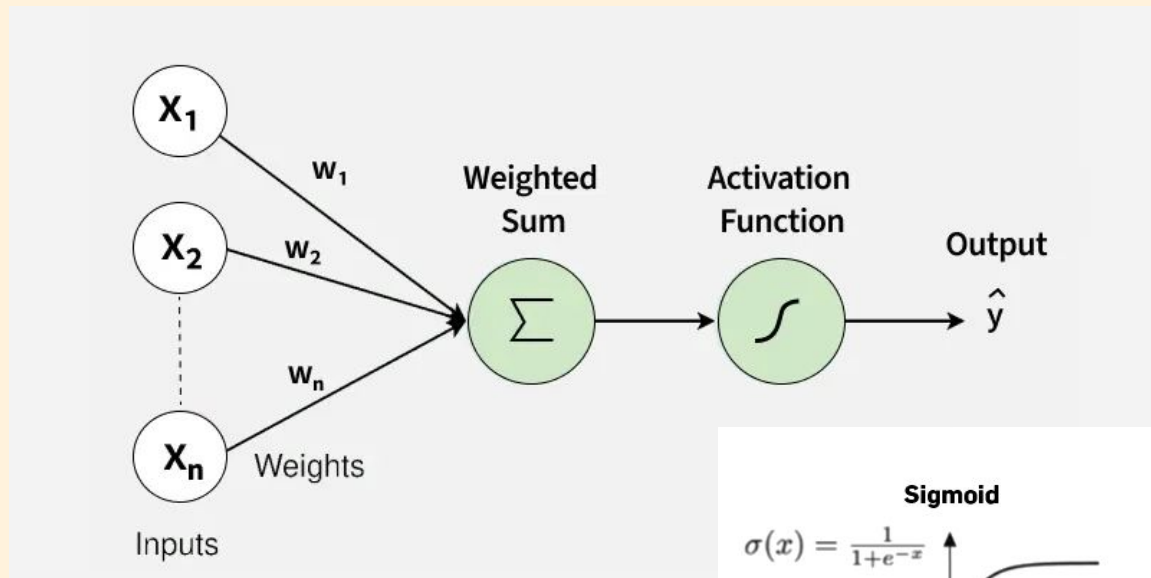


Input: $\langle 3, 5, 4 \rangle$
Weights: $\langle 2, 1, 2 \rangle$
Bias: -22
Activation: ReLU

Weighted sum: -3
 $\text{relu}(-3) = 0$



Example: Perceptron

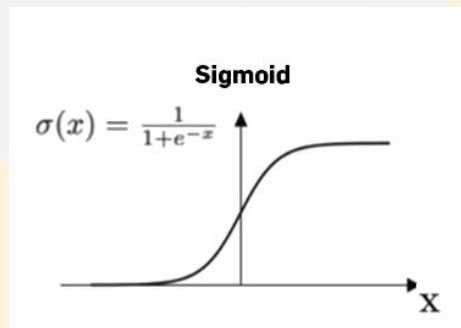


Input: $\langle 3, 5, 4 \rangle$

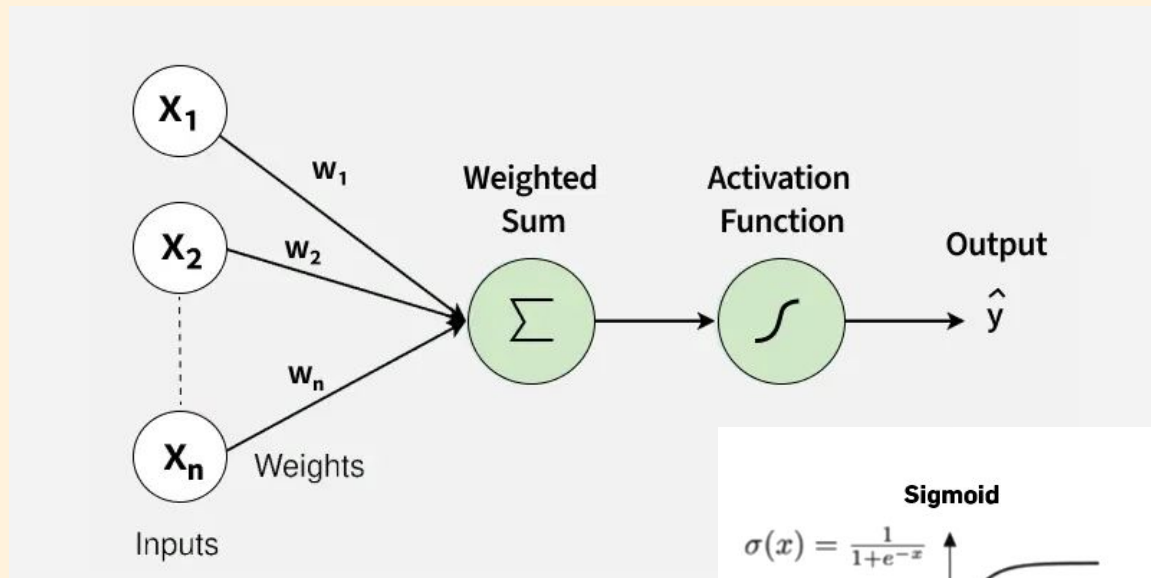
Weights: $\langle 2, 1, 2 \rangle$

Bias: -22

Activation: Sigmoid



Example: Perceptron



Input: $\langle 3, 5, 4 \rangle$

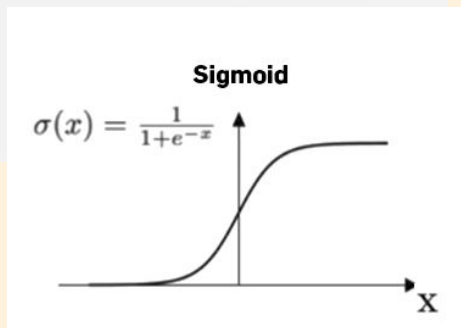
Weights: $\langle 2, 1, 2 \rangle$

Bias: -22

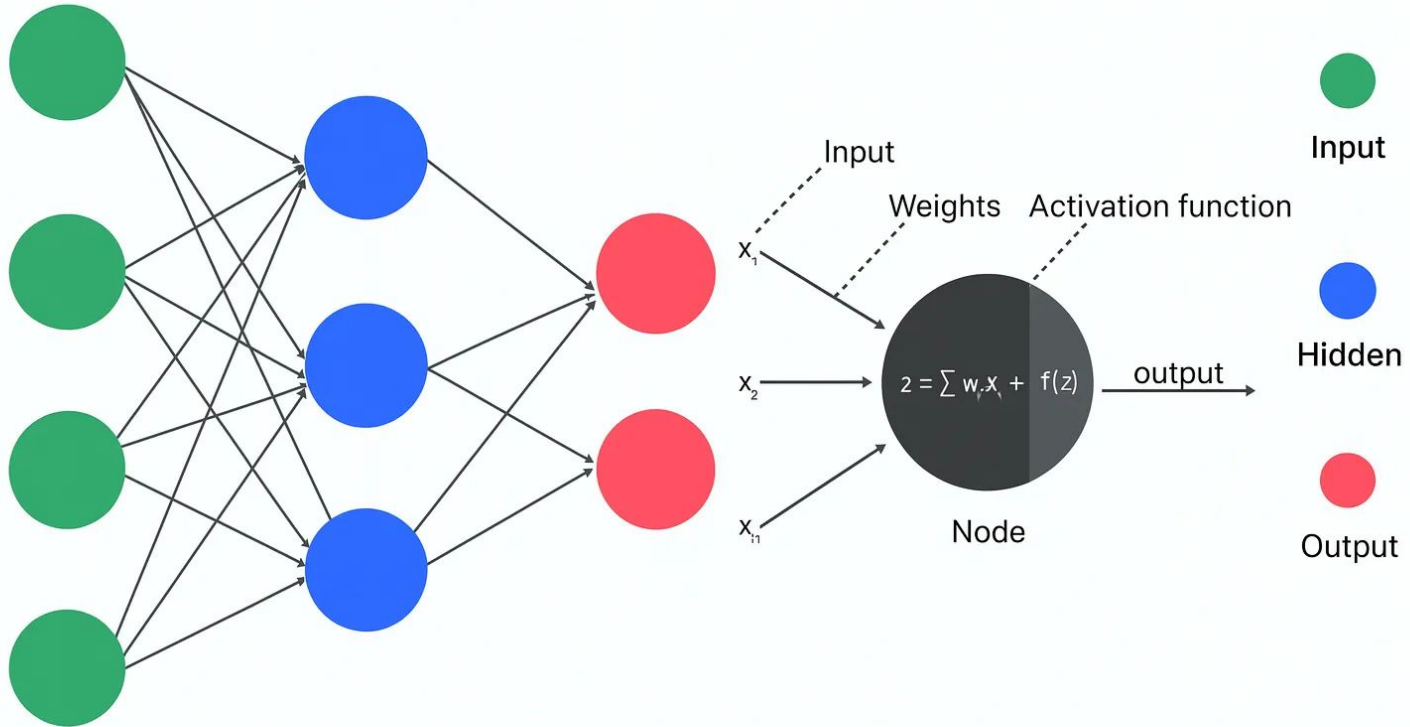
Activation: Sigmoid

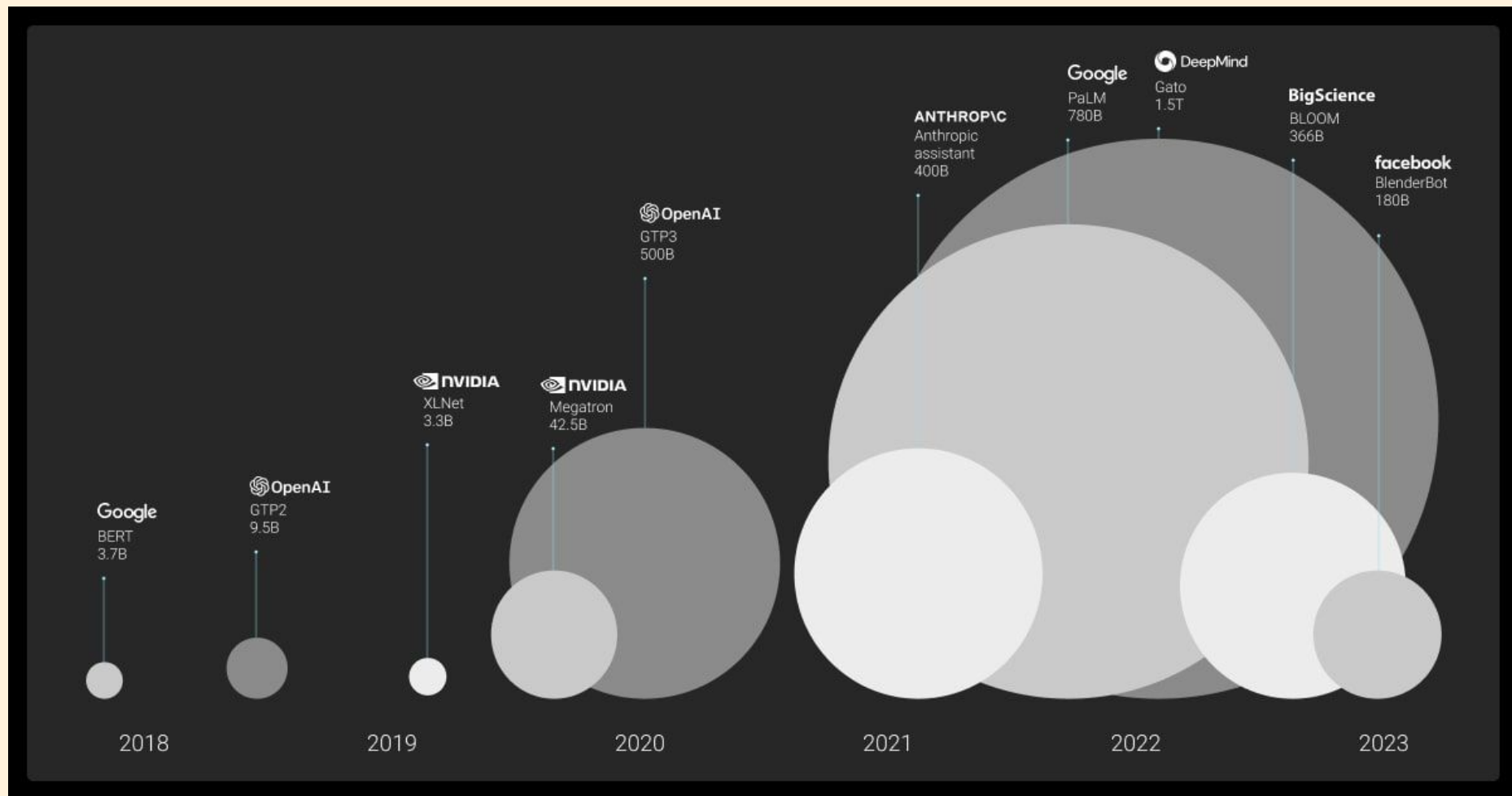
Weighted sum: -3

$\text{sigmoid}(-3) = 0.0474$



Neural Network Architecture

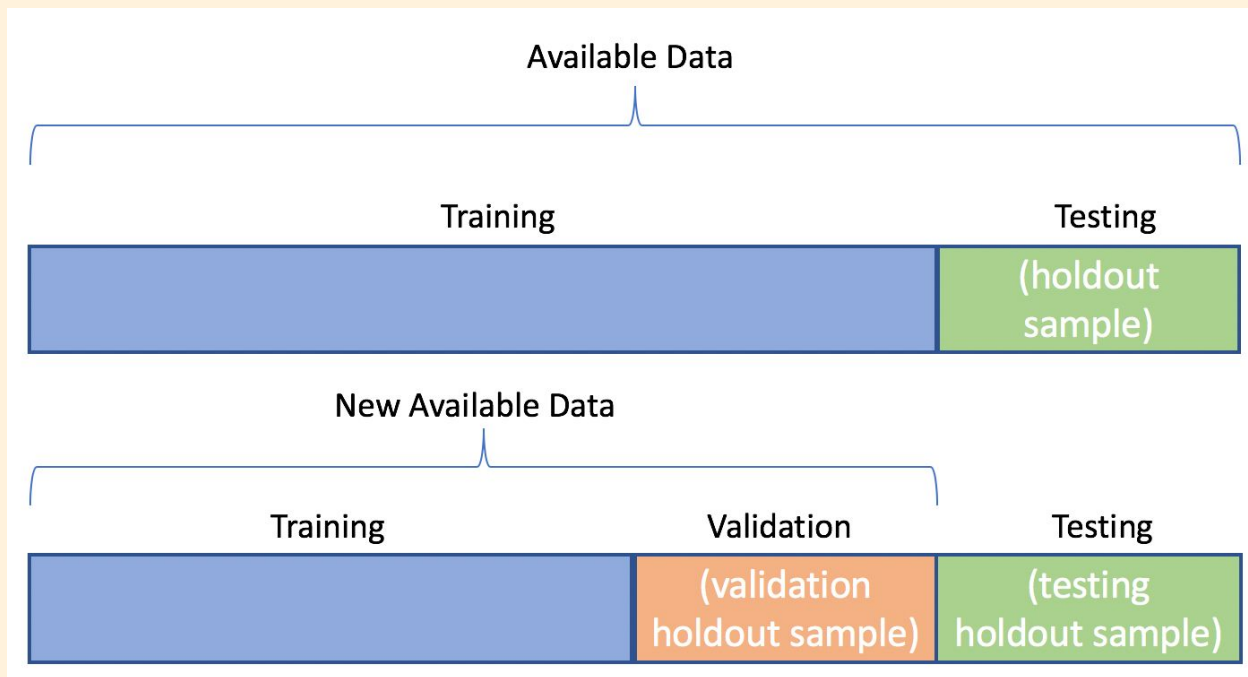




Today's Lecture

1. Neural Networks
2. **Training**
3. The Transformer

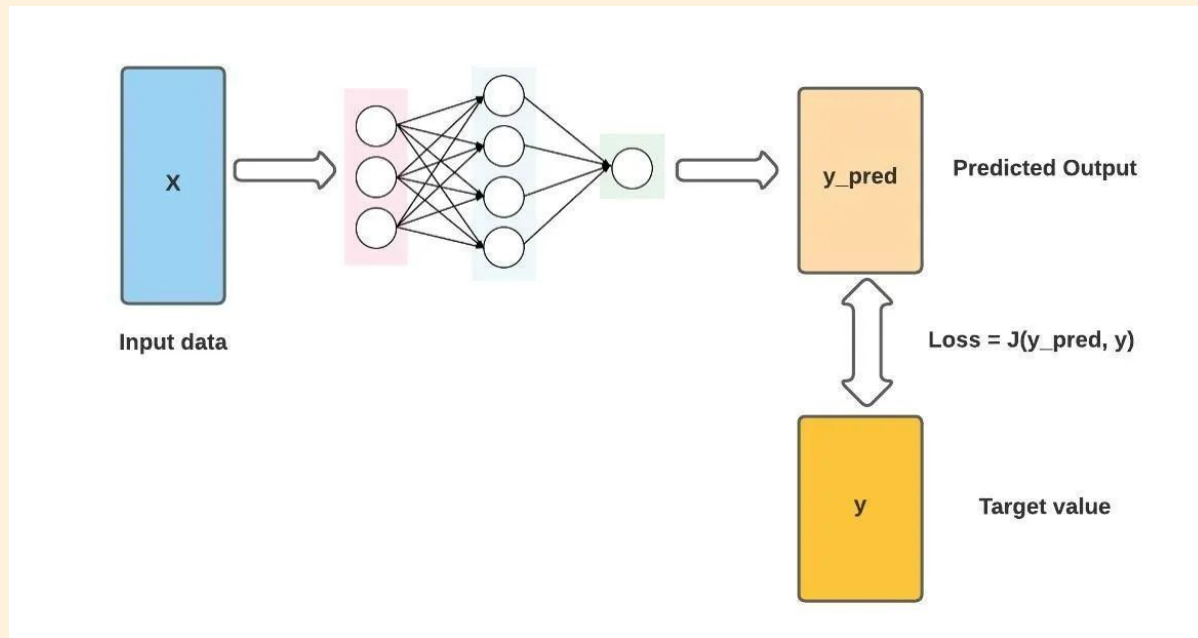
Train, Val, Test



The Loss Function

A scalar value measuring the discrepancy between prediction and ground truth.

Cost function:
average loss over
entire training set

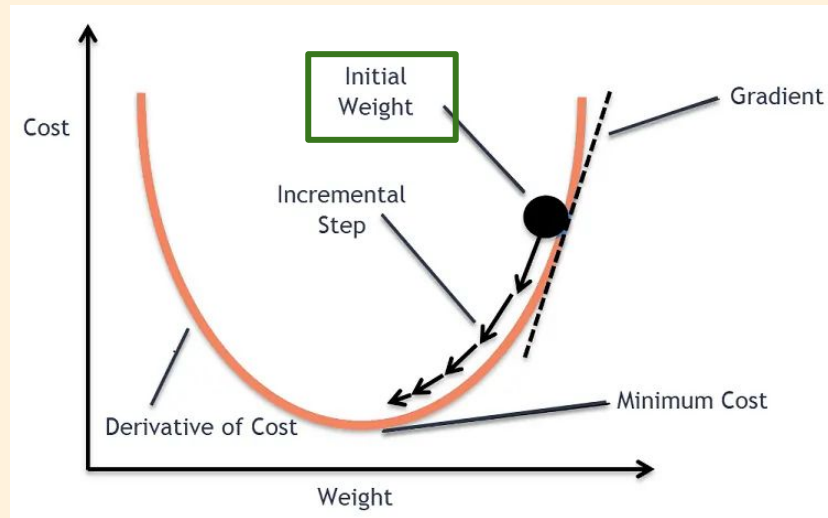


Gradient Descent

- An iterative optimization algorithm used in machine learning to find the minimum of a cost or loss function
 - Optimization problem in a high-dimensional landscape
- All training pipelines use gradient descent/ascent
 - Supervised fine-tuning
 - Instruction tuning
 - Preference optimization

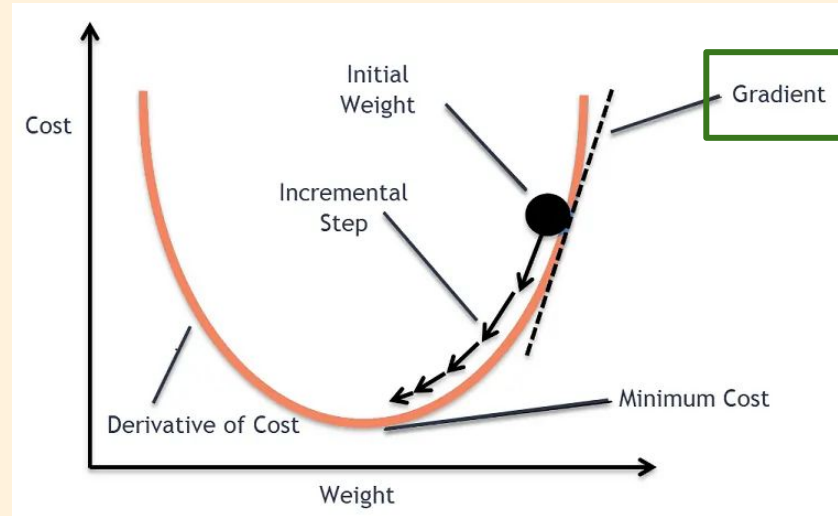
Gradient Descent

1. The algorithm begins with initial values for the model's parameters.



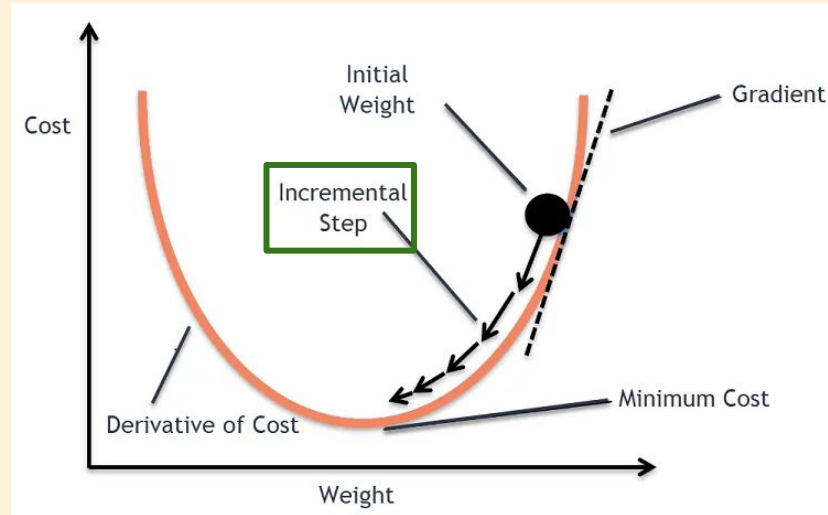
Gradient Descent

2. It computes the gradient (slope) of the cost function, which indicates the direction of the steepest increase.



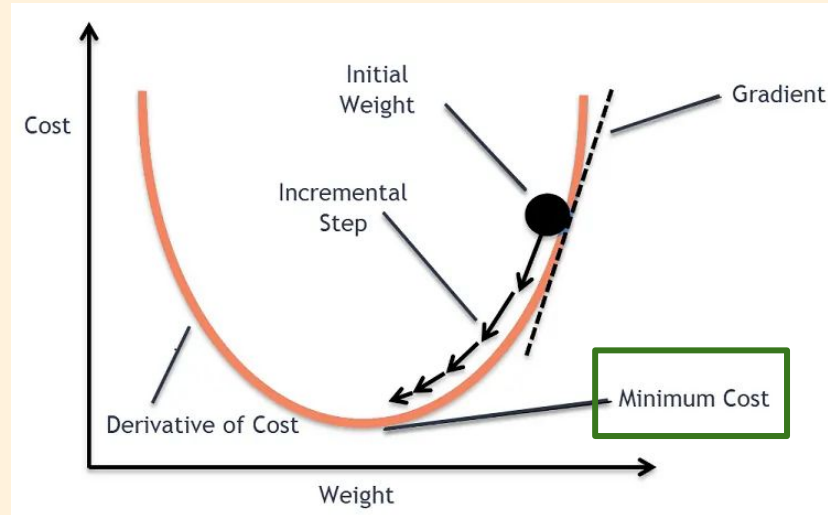
Gradient Descent

3. It then updates the parameters by taking a small step in the opposite direction of the gradient (the negative gradient), which is the direction of steepest decrease.



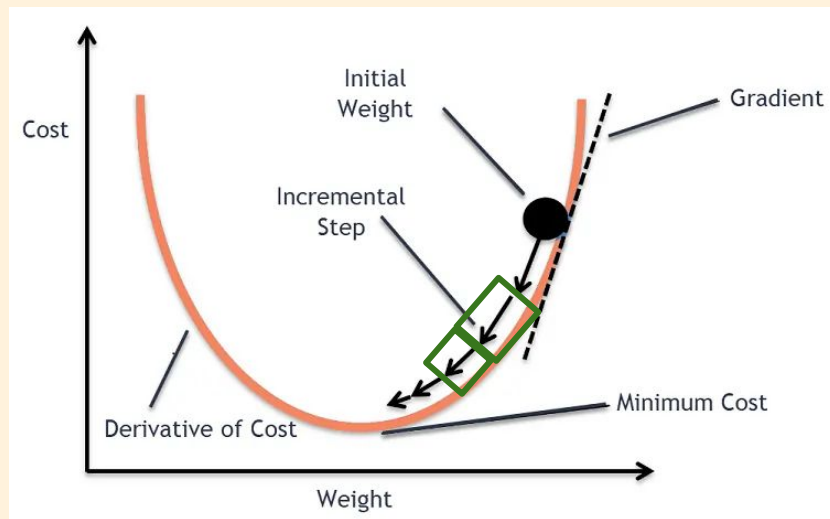
Gradient Descent

4. This process repeats, with each step getting smaller as it approaches the minimum, until the model converges (the loss stops changing significantly).



Learning Rate

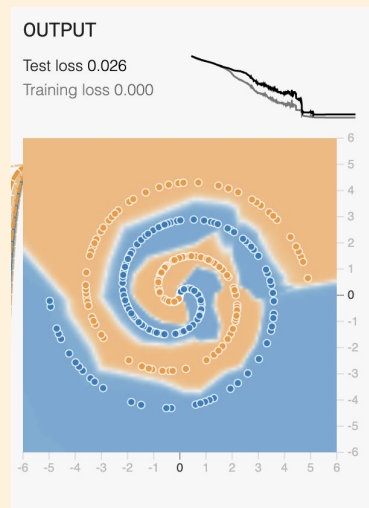
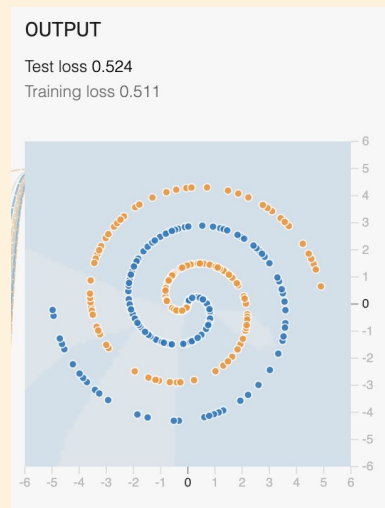
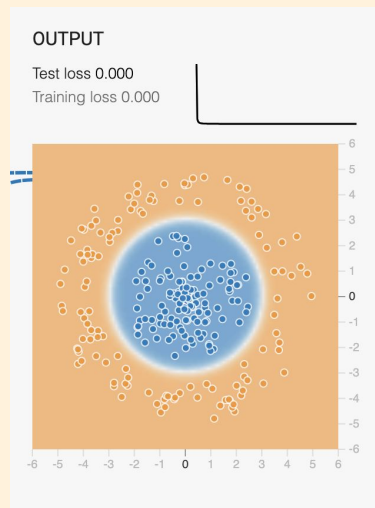
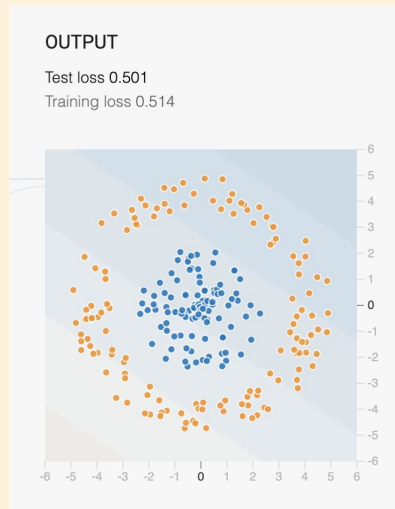
A parameter that controls how much the weights are adjusted with respect to the gradient of the loss function during training. It determines **the step size taken towards the minimum** of a loss function



Activity (10 min)

Go to playground.tensorflow.org

Adjust features, layers, neurons, activations, and learning rate.



Backpropagation

How are you supposed to determine how much each weight contributes to the final error?

Backpropagation is the method for computing those gradients efficiently.

Backpropagation

A neural network is a chain of functions:

input → *layer 1* → *layer 2* → ... → *output* → *loss*.

Forward pass: feed input through the network, layer by layer, computing activations, until you get an output and a loss value.

Backward pass: differentiates the loss function to compute how each individual parameter of the network contributes to the overall error for a single input.

Backpropagation

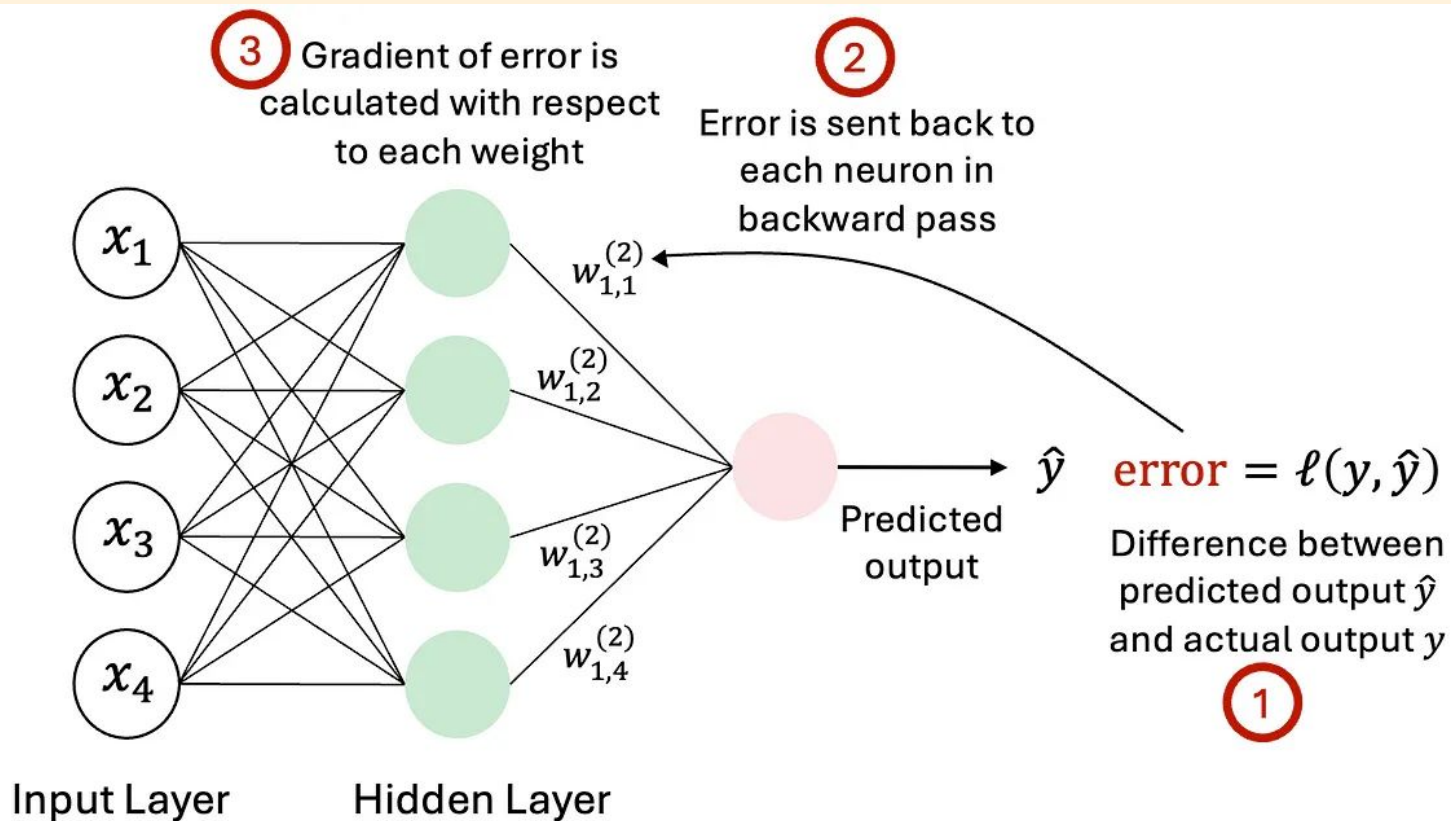
Compute how much the loss changes with respect to the output
($\partial L / \partial \text{output}$)

Use the chain rule to iteratively calculate the gradient of the loss through each layer:

$$\begin{aligned} & \partial L / \partial (\text{Layer}_i \text{ input}) \\ &= \partial L / \partial (\text{Layer}_i \text{ output}) \cdot \partial (\text{Layer}_i \text{ output}) / \partial (\text{Layer}_i \text{ input}) \end{aligned}$$

Layer i 's input = Layer $i-1$'s output

→ can iterate backwards through L-1, L-2, L-3, ... Layer 1



The training loop

1. A **forward pass**, making predictions on training data.
2. A **loss function** measures the error of the model's predictions during that forward pass.
3. **Backpropagation** of error, or a backward pass, to calculate the partial derivatives of the loss function.
4. **Gradient descent**, to update model weights.

Activity (45 min)

Watch the following videos:

- <https://www.3blue1brown.com/lessons/gradient-descent/>
- <https://www.3blue1brown.com/lessons/backpropagation/>

Complete ***gradient_descent.py*** (on the class website)

Quiz Code: **W34JK9**

Discussion

1. Look up what “local minima” are. How can you modify training to escape a local minimum?
2. If training loss goes down but test loss rises, what is happening? What are 2–3 actions you could take, and what outcome would you expect from each?
3. If both training and test loss don’t go down, what is happening? What are 2–3 actions you could take, and what outcome would you expect from each?

Today's Lecture

1. Neural Networks
2. Training
- 3. The Transformer**

The transformer

A specific kind of network architecture. It is still a feedforward neural network, but based on attention.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

Components of the Transformer

1. **Token embedding converts text into continuous vectors**
2. Positional encoding introduces sequence order
3. Self-attention layers compute token-to-token relationships
4. Feed-forward layers transform representations
5. Stacked blocks enable deep hierarchical features

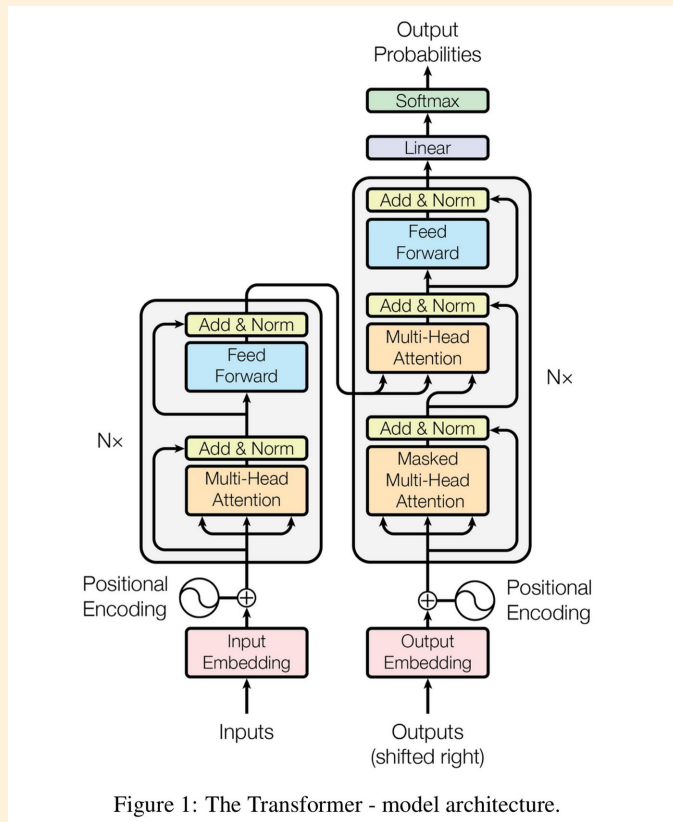
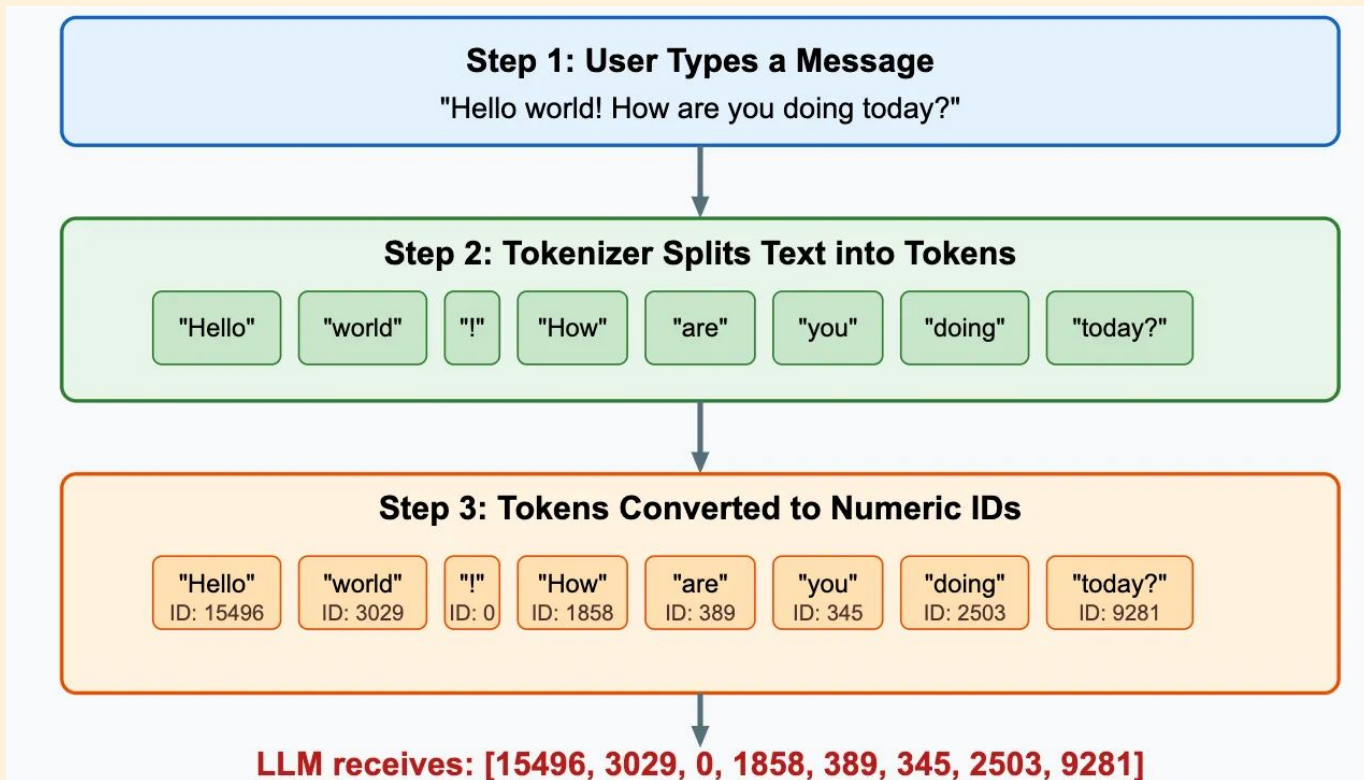


Figure 1: The Transformer - model architecture.

Raw Text to Tokens



Raw Text to Tokens

How is text split into tokens? Via a *tokenization algorithm*

- “unbelievable” → “un | believ | able”
- “GPT-5?” → “GPT”, “-”, “5”, “?”
- Emojis → each emoji is usually a token
- URLs → often split into many tokens

Why a good tokenization algorithm matters for efficiency.

- Tokens too short: text is broken up into too many pieces
- Tokens too long: the model can't generalize to unseen words

Text to Tokens: Exercise

Exercise: How does each tokenization scheme handle the following words

Training words: “burning”, “satisfactory”, “information”

Word-Level	Subword	Character-level
burning satisfactory information	burn #ing satis #fac# #tory inform #a# #tion	b u r n i g s ... etc.

1. Which vocabulary produces the fewest tokens overall for the training words?
2. Which vocabulary is the most flexible for unseen words?

Text to Tokens: Exercise

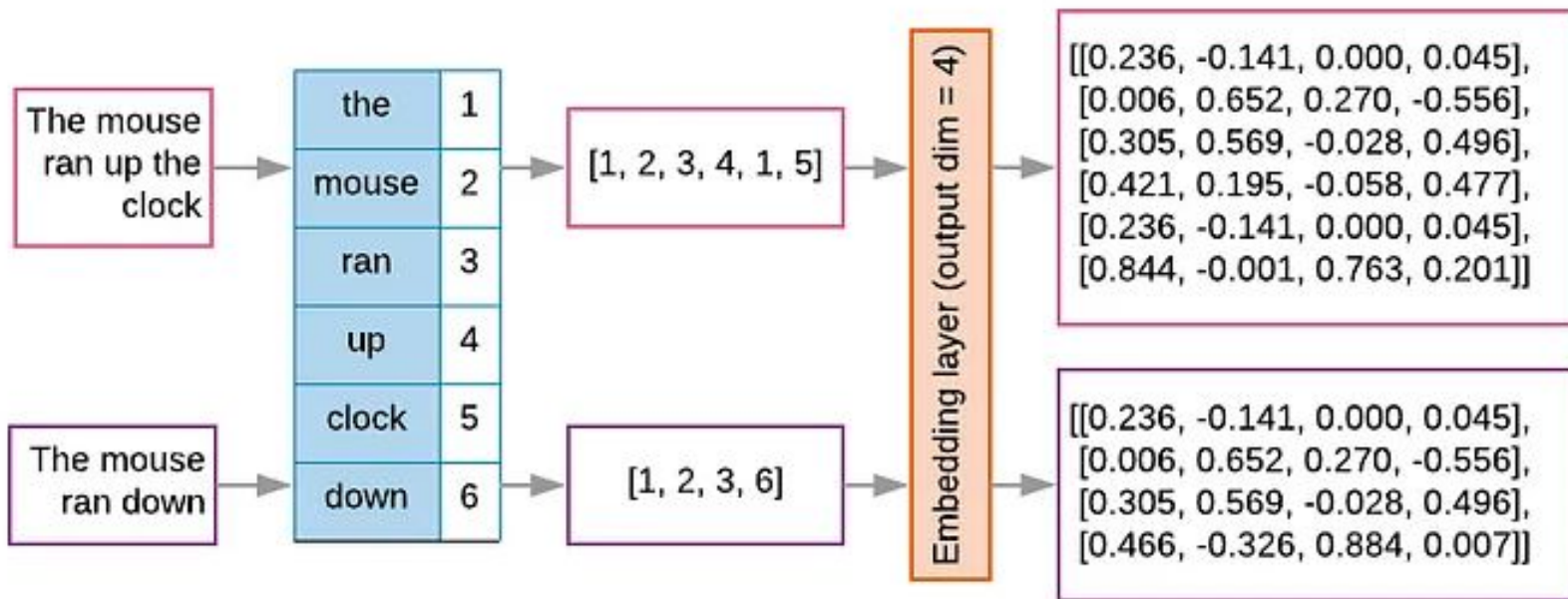
Exercise: How does each tokenization scheme handle the following words

Unseen words: “satisfaction”, “informing”

Word-Level	Subword	Character-level
burning satisfactory information	burn #ing satis #fac# #tory inform #a# #tion	b u r n i g s ... etc.

1. Which vocabulary produces the fewest tokens overall for the unseen words?
2. Which vocabulary is the least efficient for unseen words?

Embeddings



Embeddings

What do embeddings capture?

- Semantic similarity
- Syntactic roles
- Frequency-based structure

Why are embeddings useful?

- Numerical representation of language
- Words used similarly are close in vector space

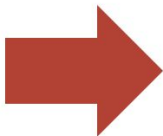
Embeddings are learned from next-token prediction during LLM training

Embeddings: Exercise

Try to build a lower dimensional embedding

Vocabulary:

Man, woman, boy,
girl, prince,
princess, queen,
king, monarch



	Femininity	Youth	Royalty
Man			
Woman			
Boy			
Girl			
Prince			
Princess			
Queen			
King			
Monarch			

Each word gets a
1x3 vector

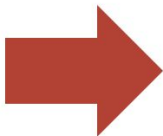
Similar words...
similar vectors

Embeddings: Exercise

Try to build a lower dimensional embedding

Vocabulary:

Man, woman, boy,
girl, prince,
princess, queen,
king, monarch



	Femininity	Youth	Royalty
Man	0	0	0
Woman	1	0	0
Boy			
Girl			
Prince			
Princess			
Queen			
King			
Monarch			

Each word gets a
1x3 vector

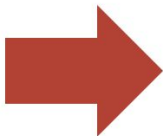
Similar words...
similar vectors

Embeddings: Exercise

Try to build a lower dimensional embedding

Vocabulary:

Man, woman, boy,
girl, prince,
princess, queen,
king, monarch



	Femininity	Youth	Royalty
Man	0	0	0
Woman	1	0	0
Boy	0	1	0
Girl	1	1	0
Prince	0	1	1
Princess	1	1	1
Queen	1	0	1
King	0	0	1
Monarch	0.5	0.5	1

Each word gets a
1x3 vector

Similar words...
similar vectors

Step 2: Embeddings

We can't simply use “femininity”, “youth”, or “royalty” to describe all words in the English language

More on embeddings:

- Each dimension doesn't measure a clear construct; it's quite complex
- Common dimensionality is 300-700
- But, research shows that enough is captured to do “word math” with embeddings
 - *“king – man + woman $\approx$ queen”*
 - *“doctor – hospital + school $\approx$ teacher”*

Components of the Transformer

1. Token embedding converts text into continuous vectors
2. **Positional encoding introduces sequence order**
3. Self-attention layers compute token-to-token relationships
4. Feed-forward layers transform representations
5. Stacked blocks enable deep hierarchical features

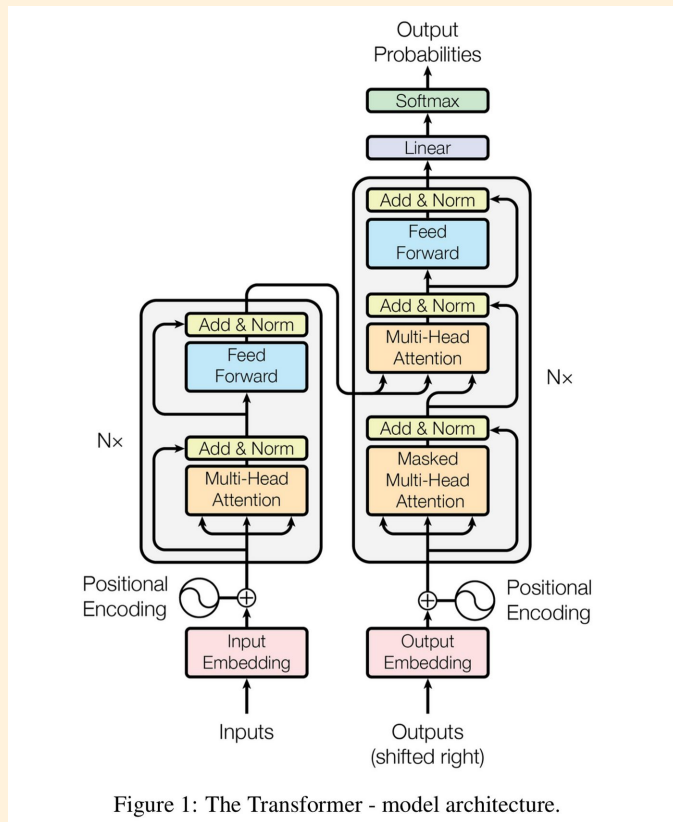


Figure 1: The Transformer - model architecture.

Positional Encodings

Transformers
process tokens in
parallel

Positional encodings
allow transformers
to know the order of
tokens

Sequence	Index of token, k	Positional Encoding Matrix with $d=4$, $n=100$			
		$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

Components of the Transformer

1. Token embedding converts text into continuous vectors
2. Positional encoding introduces sequence order
3. **Self-attention layers compute token-to-token relationships**
4. Feed-forward layers transform representations
5. Stacked blocks enable deep hierarchical features

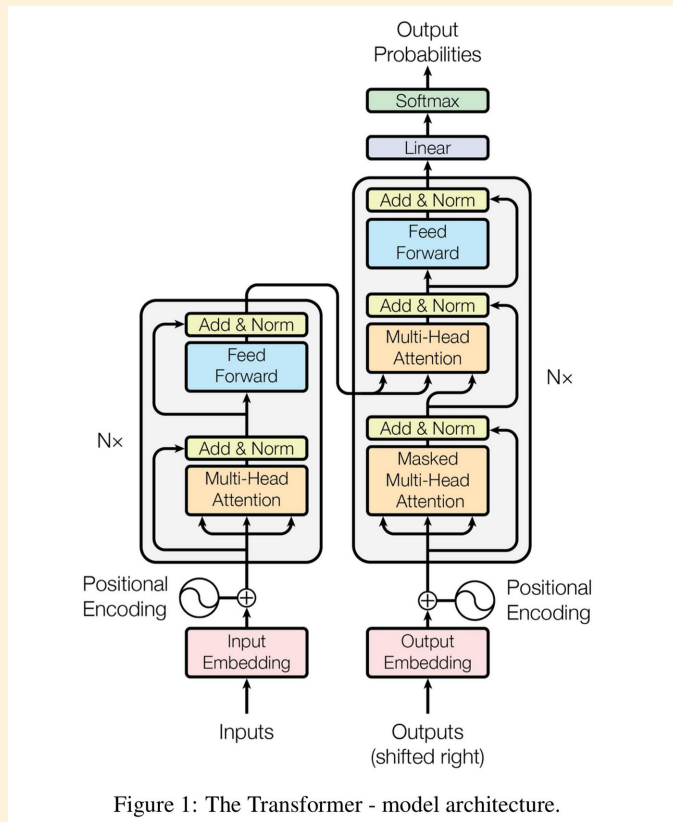


Figure 1: The Transformer - model architecture.

Problem with static word embeddings

They are static! The embedding for a word doesn't reflect how its meaning changes in context.

The chicken didn't cross the road because it was too tired

What is the meaning represented in the static embedding for "it"?

Contextual Embeddings

- Intuition: a representation of meaning of a word should be different in different contexts!
- Contextual Embedding: each word has a different vector that expresses different meanings depending on the surrounding words
- How to compute contextual embeddings?
 - Attention!!

Contextual Embeddings

The chicken didn't cross the road because it

What should be the properties of "it"?

The chicken didn't cross the road because it was too tired

The chicken didn't cross the road because it was too wide

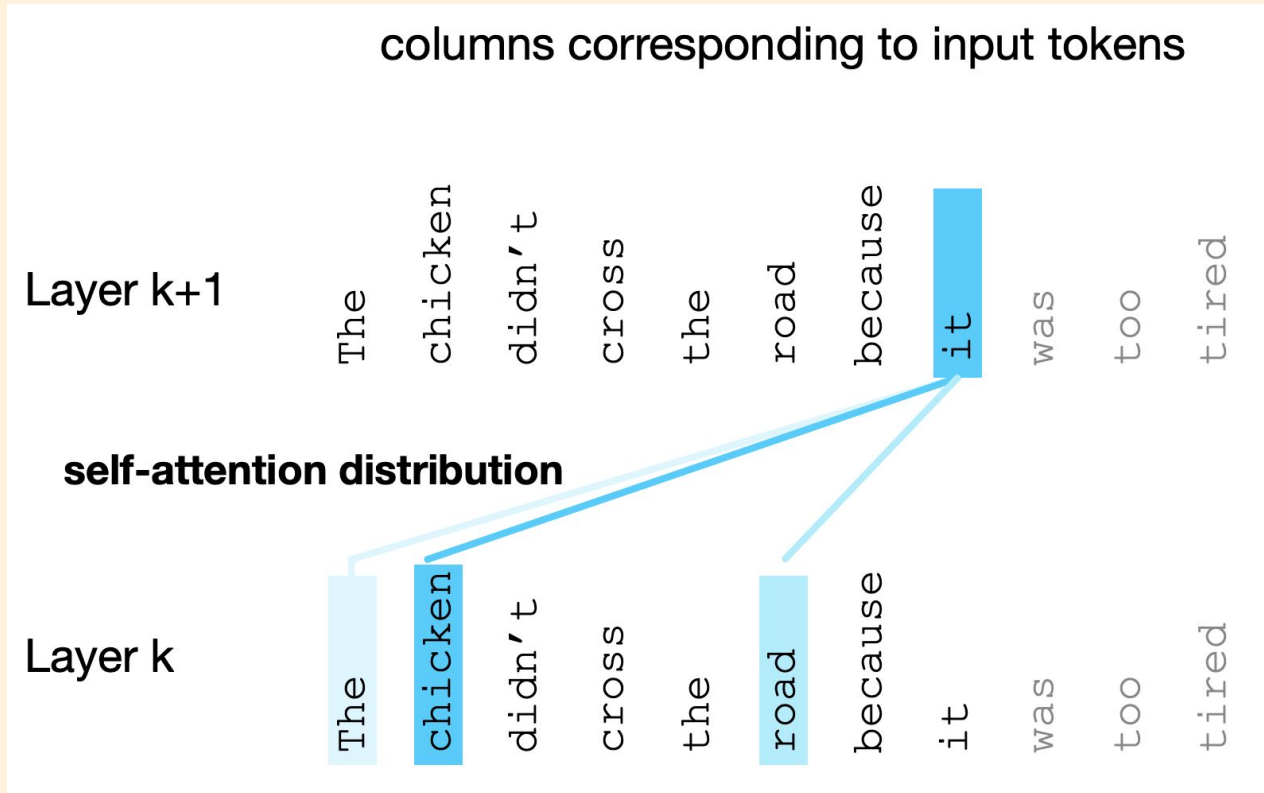
At this point in the sentence, it's probably referring to either the chicken or the street

Intuition of attention

Build up the contextual embedding from a word by selectively integrating information from all the neighboring words

We say that a word "attends to" some neighboring words more than others

Intuition of attention



So, what is attention?

A mechanism for helping compute the **contextual embedding for a token** by selectively attending to and integrating information from surrounding tokens (at the previous layer).

In practice: computed using a weighted sum of vectors.

Input
sequence

'the', 'train', 'left', 'the', 'station', 'on', 'time'

Token
vectors



Vectors learned during training

Attention score
 $A(i,j)$

= dot(v_i , v_j)

*= how much
token i attends
to token j*

	the	train	left	the	station	on	time
the	1.0	0.3	0.1	0.5	0.2	0.1	0.1
train	0.3	1.0	0.6	0.3	0.8	0.1	0.2
left	0.1	0.6	1.0	0.1	0.6	0.1	0.1
the	0.5	0.3	0.1	1.0	0.3	0.1	0.2
station	0.2	0.8	0.6	0.3	1.0	0.2	0.2
on	0.1	0.1	0.1	0.1	0.2	1.0	0.5
time	0.1	0.2	0.1	0.2	0.2	0.5	1.0

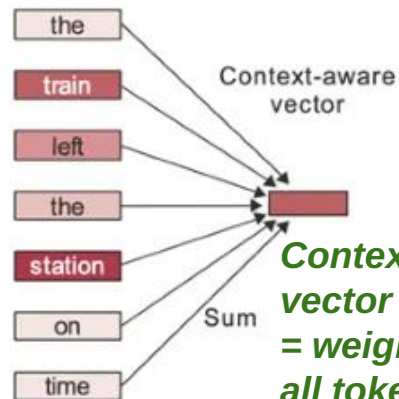
Attention scores

Scores for
"station"

0.2
0.8
0.6
0.3
1.0
0.2
0.2

Softmax,
scaling, and
multiplication

Weighted
token vectors



*Contextualized
vector for "station"
= weighted sum of
all token vectors by
attention to
"station"*

Core of the Transformer is Self Attention as in the original paper: “**Attention** is all you need”.

arxiv.org/pdf/1706.03762.pdf

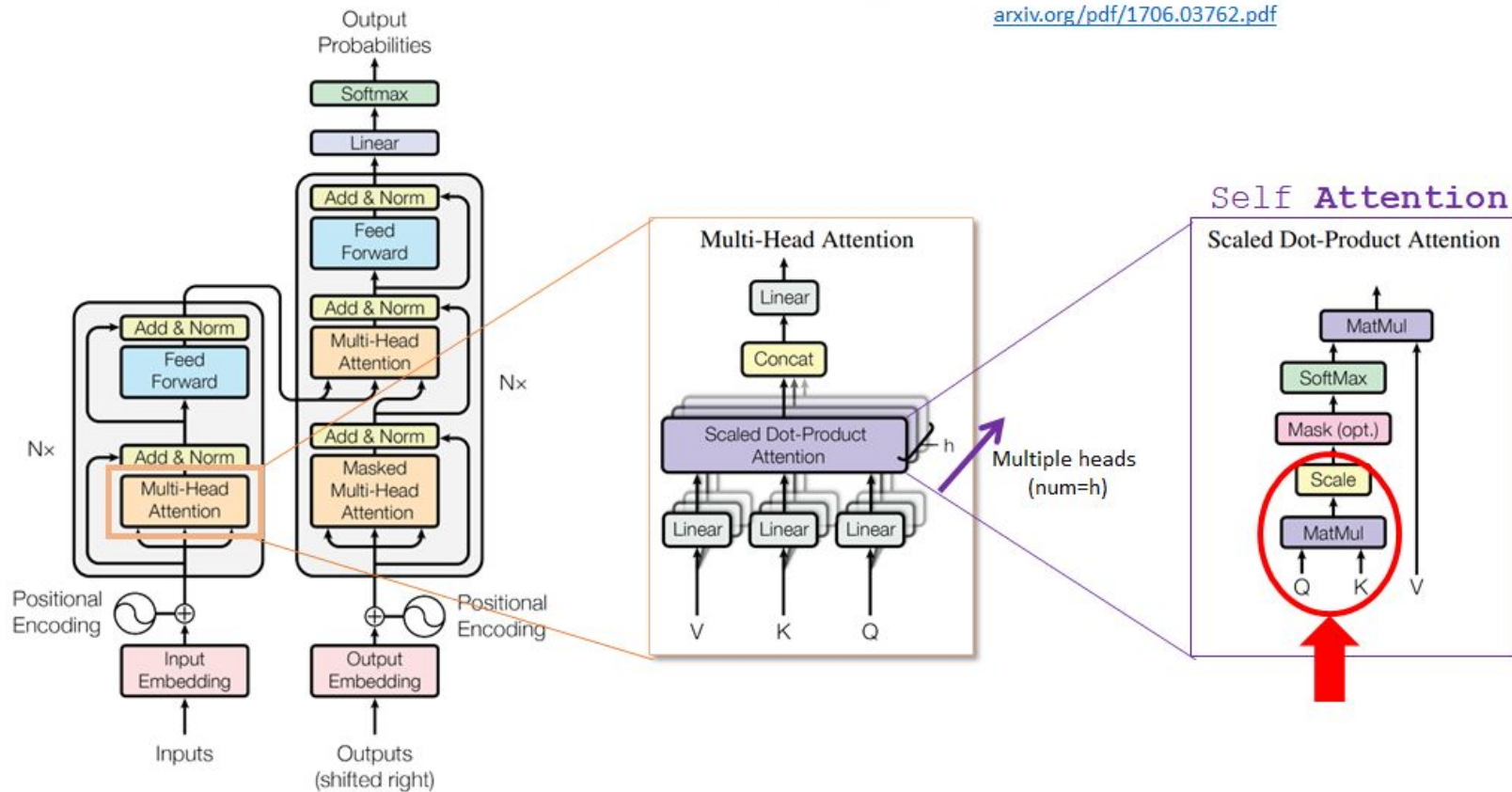


Figure 1: The Transformer - model architecture.

Components of the Transformer

1. Token embedding converts text into continuous vectors
2. Positional encoding introduces sequence order
3. Self-attention layers compute token-to-token relationships
4. **Feed-forward layers transform representations**
5. Stacked blocks enable deep hierarchical features

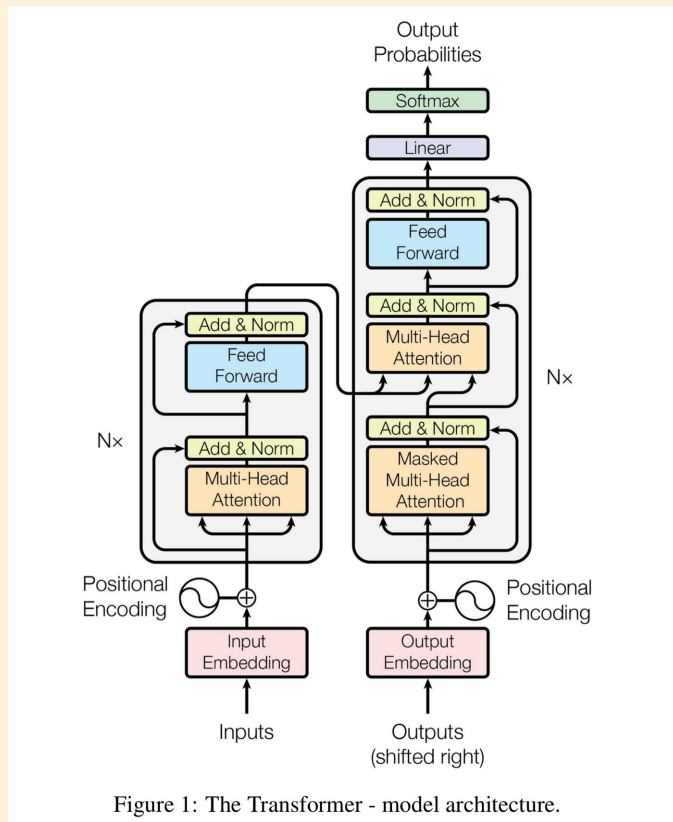


Figure 1: The Transformer - model architecture.

Components of the Transformer

1. Token embedding converts text into continuous vectors
2. Positional encoding introduces sequence order
3. Self-attention layers compute token-to-token relationships
4. Feed-forward layers transform representations
5. **Stacked blocks enable deep hierarchical features**

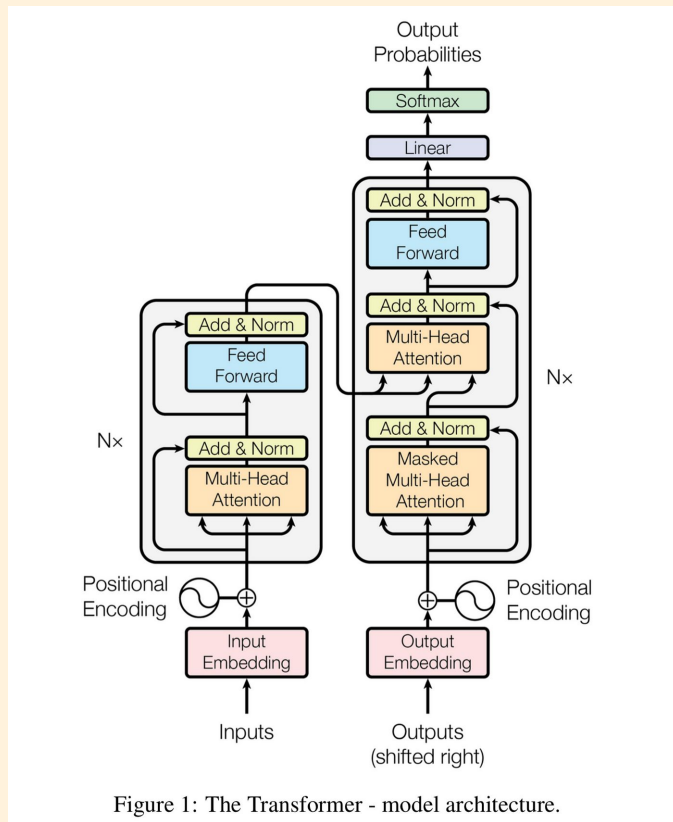


Figure 1: The Transformer - model architecture.

Discussion

1. Explain the architecture of a transformer. What makes it different from RNNs and LSTMs?
2. A transformer's *context window* is the number of tokens it can take as input. What happens to the number of attention scores as the context window increases?